

Seguridad y Encriptación

De la Teoría Criptográfica a la Implementación
Offline-First con Supabase.

Present By. Brayan Castro Diaz y Jorman David Noriega

Encriptación



La encriptación es una forma de codificar los datos para que solo las partes autorizadas puedan entender la información. En términos técnicos, es el proceso de convertir un texto plano legible para seres humanos en un texto incomprensible, también conocido como texto encriptado. Si usamos términos más sencillos, la encriptación coge datos legibles y los altera para que parezcan aleatorios. La encriptación requiere el uso de una clave criptográfica: un conjunto de valores matemáticos pactados tanto por el emisor como por el receptor de un mensaje encriptado.

El objetivo principal de la encriptación es: Garantizar la confidencialidad y proteger los datos frente a accesos no autorizados, tanto en tránsito como en reposo.



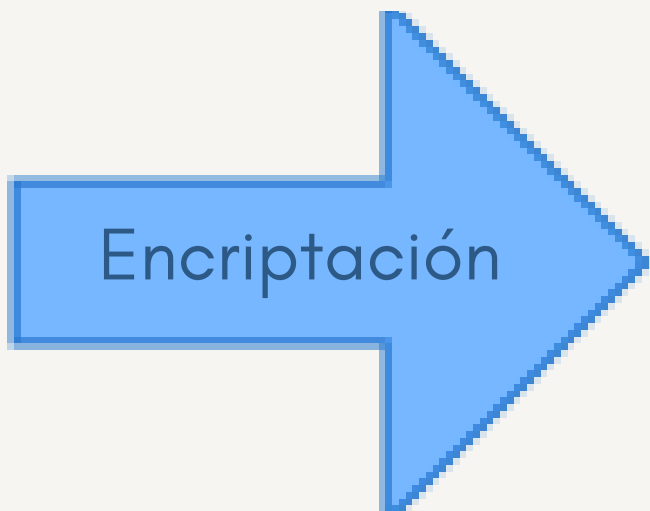
Como funciona la Criptografía

La encriptación es un proceso matemático que altera los datos al usar un algoritmo de encriptación y una clave. Imagina que Alicia envía el mensaje "Hola" a Bob, pero sustituye cada letra de su mensaje por la letra que le sigue dos lugares más adelante en el alfabeto. En lugar de "Hola," su mensaje ahora dice "Jgnnq". Afortunadamente, Bob sabe que la clave es "2" y puede descifrar su mensaje a "Hola."

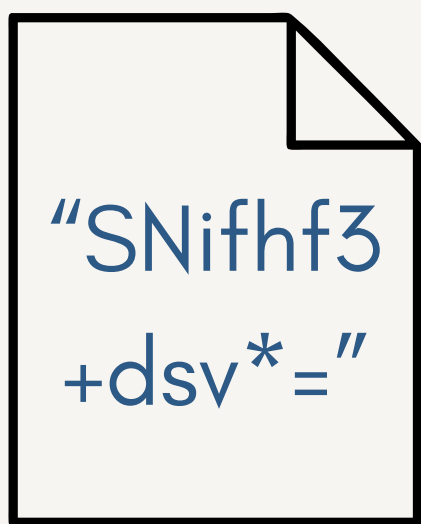


"Hello"

plaintext



Encriptación



"SNifhf3
+dsv*="

ciphertext



¿Qué es una clave en criptografía?

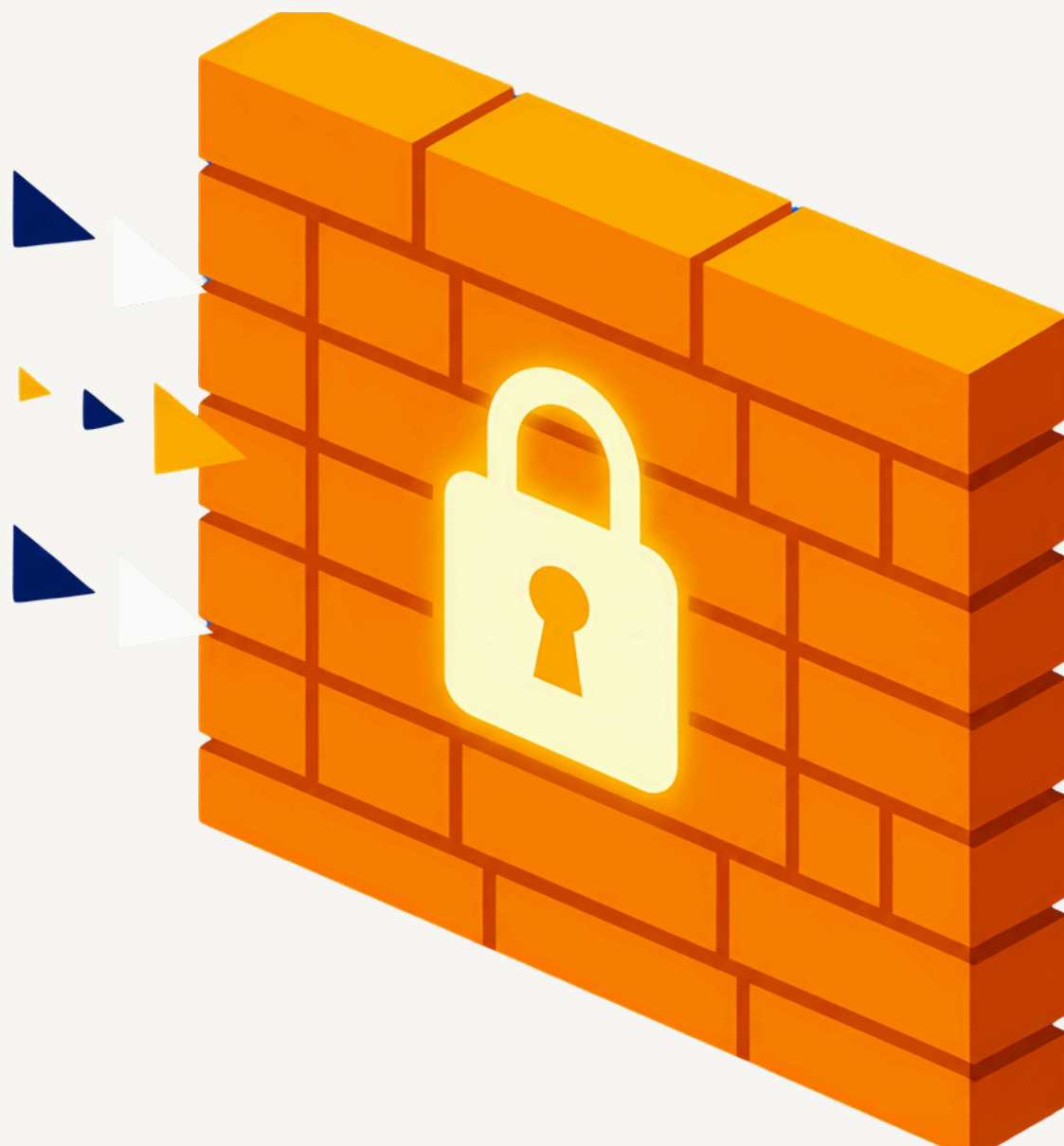
Una clave criptográfica es un valor digital (una cadena de bits o caracteres) que actúa como el parámetro fundamental para controlar el funcionamiento de un algoritmo de cifrado. Mientras que el algoritmo es el método matemático general, la clave es lo que hace que ese método sea único y seguro.

Funcionamiento Fundamental

- Transformación: El algoritmo aplica operaciones matemáticas utilizando la clave sobre el texto claro (datos originales) para transformarlo en texto cifrado (ilegible).
- Control de Acceso: Sin la clave correcta, revertir el proceso (descifrado) es computacionalmente inviable, incluso si se conoce perfectamente el algoritmo utilizado (cumpliendo con el Principio de Kerckhoffs).



Tipos de Cifrado



Cifrado Simétrico

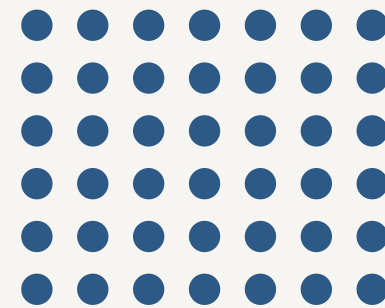
El cifrado simétrico funciona mediante la creación de una única clave compartida para cifrar y descifrar datos confidenciales. La principal ventaja del cifrado simétrico es que es simple y eficiente para proteger los datos.

texto cifrado = cifrar(texto plano, clave) y texto plano = descifrar(texto cifrado, clave)

La seguridad de todo el sistema se basa en dos elementos: la confidencialidad de la clave y la robustez del algoritmo. Si alguno de ellos se ve comprometido, los datos quedan expuestos, por lo que las claves deben generarse con una alta aleatoriedad (utilizando un generador de números pseudoaleatorios criptográficamente seguro) y protegerse cuidadosamente durante toda su vida útil.

Algoritmos: AES (Advanced Encryption Standard), DES, 3DES, ChaCha20.

Casos de uso: Cifrado de discos duros (BitLocker, FileVault), bases de datos corporativas y el cifrado de las sesiones en aplicaciones de mensajería (después de un intercambio inicial).



Cifrado Asimétrico

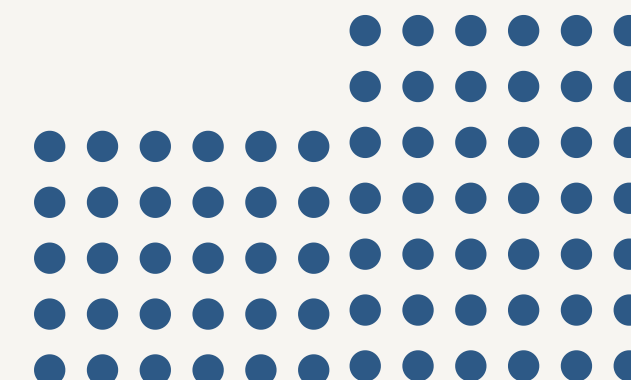
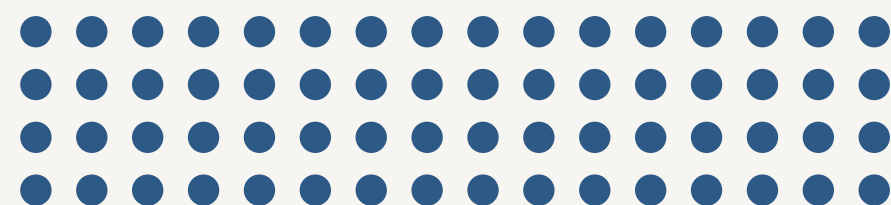
El cifrado asimétrico emplea dos claves diferentes (una pública para cifrar y otra privada para descifrar) para garantizar comunicaciones seguras sin necesidad de un intercambio previo de claves secretas. Es más seguro pero menos eficiente.

texto cifrado = cifrar(texto plano, clave pública) y texto plano = descifrar(texto cifrado, clave privada).

La seguridad depende de la confidencialidad de la clave privada y el uso de marcos PKI, requiriendo claves de mayor longitud que el cifrado simétrico para ser robustas.

Algoritmos / Esquemas: RSA, ECC (Elliptic Curve Cryptography) e Infraestructura de Clave Pública (PKI).

Casos de uso: Confidencialidad en mensajes, autenticación de identidad, firmas digitales y sistemas de cifrado híbrido junto al cifrado simétrico.



¿Qué es un ataque de fuerza bruta en encriptación?

Un ataque de fuerza bruta es cuando un atacante que no conoce la clave de descryptación intenta determinar la clave haciendo millones o miles de millones de conjeturas. Los ataques de fuerza bruta son mucho más rápidos con los ordenadores modernos, por lo que la encriptación tiene que ser extremadamente fuerte y compleja.

mayoría de los métodos modernos de encriptación, junto con las contraseñas de alta calidad, son resistentes a los ataques de fuerza bruta, aunque pueden llegar a ser vulnerables a estos ataques en el futuro, a medida que los ordenadores se vuelven cada vez más potentes. Las contraseñas débiles siguen siendo susceptibles de sufrir ataques de fuerza bruta.



¿Por qué es importante la encriptación?

Privacidad:



- Protege datos (en tránsito o reposo) para que solo los lea el destinatario legítimo.
- Evita la interceptación de atacantes, redes publicitarias y gobiernos.

Seguridad:



- Previene fugas de datos y protege la información ante robos o pérdida de dispositivos (ej. discos duros cifrados).

Integridad:



- Garantiza que los datos transmitidos por internet no sufran modificaciones ni ataques en el camino (Man-in-the-Middle).

Cumplimiento Normativo:



- Obligatorio por normativas internacionales para proteger la información de los usuarios (HIPAA, PCI-DSS, RGPD).



Protección y Encriptación de Datos en Reposo

Tu Vacuna PAI es una aplicación móvil diseñada para apoyar la gestión y el registro de información relacionada con la vacunación, especialmente en escenarios donde la conectividad a Internet puede ser limitada o intermitente.

Uno de los pilares de la aplicación es su motor offline-first. Decidimos implementar este enfoque porque el proceso de vacunación no puede depender completamente de tener una conexión permanente a Internet. El personal encargado debe poder consultar información, registrar atenciones y continuar trabajando incluso cuando no exista conexión con el servidor.

Por esta razón, la aplicación cuenta con una base de datos local que permite almacenar temporalmente la información directamente en el dispositivo. De esta manera, el sistema puede seguir funcionando de forma autónoma y posteriormente sincronizar los datos con el servidor cuando la conectividad esté disponible nuevamente.

Objetivos

Objetivo General (Nivel: Crear / Desarrollar):

- Desarrollar una capa de persistencia local segura en la aplicación móvil Tu Vacuna PAI para proteger la información clínica y de vacunación en reposo frente a extracciones físicas o robos del dispositivo.

Objetivos Específicos (Nivel: Implementar y Verificar):

- Implementar el cifrado transparente a nivel de archivo mediante la integración de SQLite3MultipleCiphers y una gestión segura de claves en el almacenamiento del sistema operativo.
- Verificar la eficacia del aislamiento de datos mediante pruebas automatizadas de regresión y análisis empírico de bytes en disco (hexdump).



Vulnerabilidad del Almacenamiento Local Tradicional

- ✓ **SQLite Estándar:** Los datos, esquemas e índices residen en texto plano.
- ✓ **Cabecera legible:** SQLite format 3 visible con herramientas básicas.
- ✓ **Riesgo:** Extracción del archivo por pérdida, robo o acceso USB al almacenamiento del dispositivo.
- ✓ **Necesidad:** El archivo en reposo debe ser indistinguible de ruido aleatorio sin la clave de descifrado.



Cifrado a Nivel de Archivo con SQLite3MC

Decisión

Integración de SQLite3MultipleCiphers (sqlite3mc) en el binario nativo.

Transparencia

Drift, transacciones y consultas SQL operan de forma idéntica; no requiere lógica de cifrado en repositorios

Alcance

Cifra todo el archivo (datos, esquema, índices, WAL y páginas libres)

Flujo



Arquitectura Criptográfica de la Base de Datos

Generación de Clave

Clave aleatoria de 256 bits generada por CSPRNG (Random.secure()) en el primer arranque.

Almacenamiento Seguro

Persistida en el Secure Storage del SO (Keychain/Keystore); nunca en texto plano.

Parámetros Configurados

ChaCha20-Poly1305 (Cifrado autenticado AEAD).

PBKDF2-HMAC-SHA512 con 64007 rondas.

Cuatro momentos clave

El cifrado no es una sola operación, sino una cadena de responsabilidades bien delimitadas.

1

1. Compilar

SQLite con sqlite3mc

2

2. Iniciar

App y SecureStorage

3

3. Clave

Generar o recuperar

4

4. Cifrar

Motor automático

Paso 1: Compilar SQLite con soporte de cifrado

En pubspec.yaml se sustituye SQLite estándar por **SQLite3MultipleCiphers** (sqlite3mc), con el codec de cifrado integrado en el motor.

```
hooks:
  user_defines:
    sqlite3:
      source: sqlite3mc
```

Paso 2: La aplicación inicia

main.dart:52

Se crea el almacenamiento seguro que protegerá la clave:

```
final secureStorage =
  const FlutterSecureStorage();
```

main.dart:56

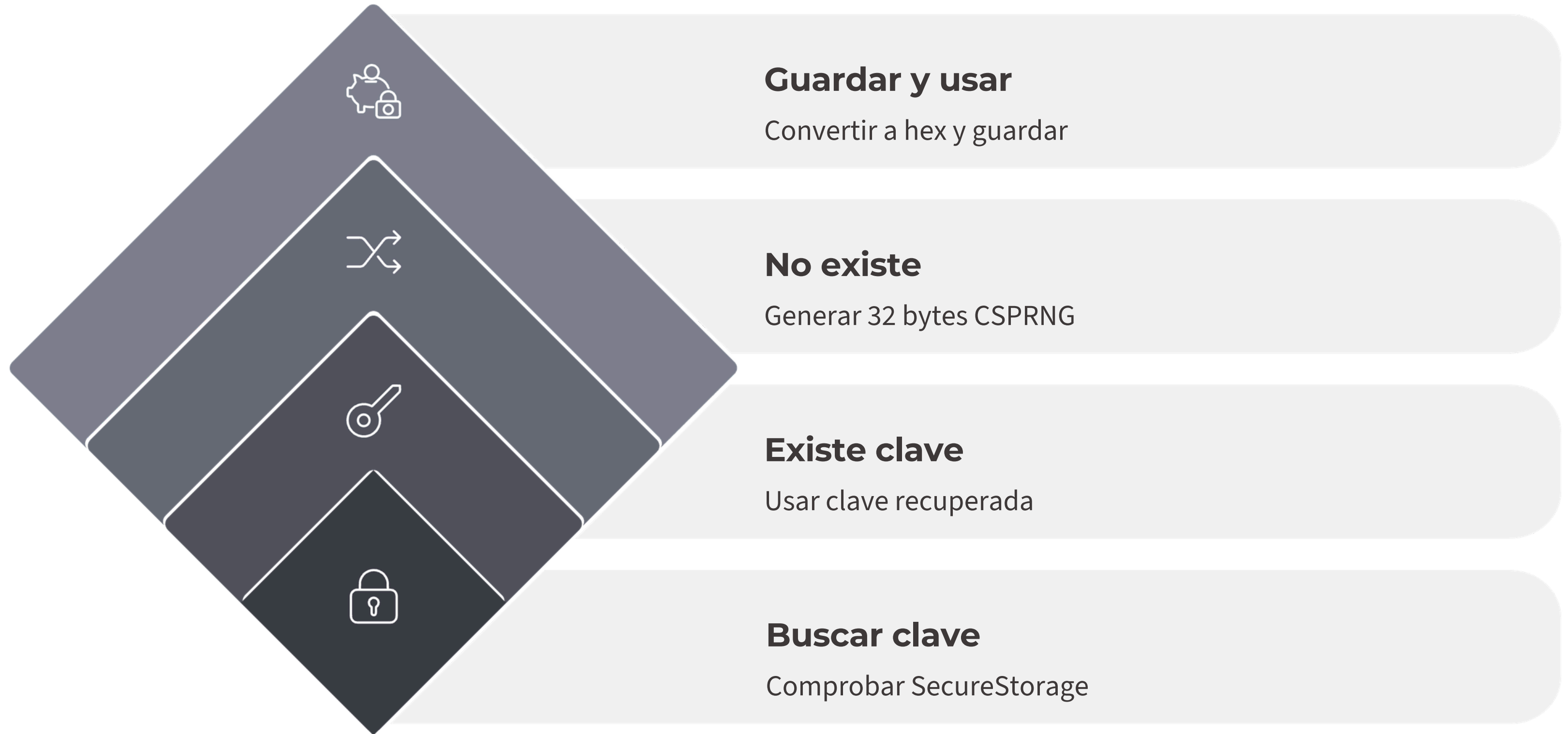
Se abre la base cifrada:

```
final appDatabase =
  await
  AppDatabase.open(secureStorage);
```

FlutterSecureStorage guarda la clave, no la base. La protección usa las credenciales del SO (Keychain / Keystore / DPAPI) y requiere el dispositivo desbloqueado.

Paso 3: Búsqueda o generación de la clave

`AppDatabase.open()` llama a `_loadOrCreateKey(storage)`, que busca `local_db_key` en `FlutterSecureStorage`.



Si es la primera ejecución, se generan **32 bytes = 256 bits** de aleatoriedad criptográfica (CSPRNG), se convierten a hexadecimal y se persisten en Secure Storage bajo `local_db_key`.

Generación de la clave (primera ejecución)

Código — app_database.dart:335-340

```
final random = Random.secure();  
final bytes = List.generate(  
  32, (_) => random.nextInt(256));  
  
final key = bytes  
  .map((b) => b.toRadixString(16)  
    .padLeft(2, '0'))  
  .join();  
  
await storage.write(  
  key: _databaseKeyName, value: key);
```

Flujo conceptual

01

Bytes aleatorios

A7 31 9C 04 ...

02

Conversión a hex

a7319c04...

03

Secure Storage

local_db_key → a7319c04...

Pasos 5 y 6: Conexión y entrega de la clave

Apertura de la conexión

Drift prepara la conexión nativa. El setup se ejecuta antes de cualquier operación SQL:

```
final connection = driftDatabase(  
  name: 'tu_vacuna_pai',  
  native: DriftNativeOptions(  
    setup: (raw) { ... },  
  ),  
);
```

PRAGMA dentro del setup

```
raw.execute("PRAGMA key = '$key'");  
raw.execute(  
  'PRAGMA cipher_memory_security = OFF');
```

- **PRAGMA key:** entrega la clave al motor.
- **cipher_memory_security = OFF:** desactiva borrado defensivo de memoria por rendimiento; no afecta al cifrado en disco.



El orden es obligatorio: primero PRAGMA key, después la configuración del cifrador.

Derivación de clave y apertura de la base

Antes de tocar las páginas, el motor aplica **PBKDF2-HMAC-SHA512** con 64007 rondas. La clave no se usa en bruto.

Parámetro	Valor	Significado
PRAGMA cipher	chacha20	ChaCha20-Poly1305 (AEAD simétrico)
PRAGMA kdf_iter	64007	PBKDF2-HMAC-SHA512, 64007 rondas

Clave correcta Archivo cifrado → descifrado por página → Drift → App	Clave incorrecta SqliteException: file is not a database
--	--

INSERT y SELECT: el motor lo hace todo

Escribir datos (INSERT)

Tu código usa SQL normal. El motor cifra la página antes de escribirla a disco:

```
Future setSyncMetadata(  
    String key, String value) {  
    return into(syncMetadata)  
        .insertOnConflictUpdate(...);  
}
```

Datos en memoria (claro) → ChaCha20+Poly1305 →
tu_vacuna_pai.sqlite cifrado.

Leer datos (SELECT)

La app hace SELECT y el flujo inverso ocurre automáticamente:

01

Archivo cifrado

Páginas de 4 KiB en disco

02

sqlite3mc descifra

Página a página en memoria

03

Drift → Dart

Datos en claro para la app



Nunca escribes decrypt(data). El descifrado lo hace el motor.

¿Qué queda protegido? Límites del modelo

Capa 1 — FlutterSecureStorage

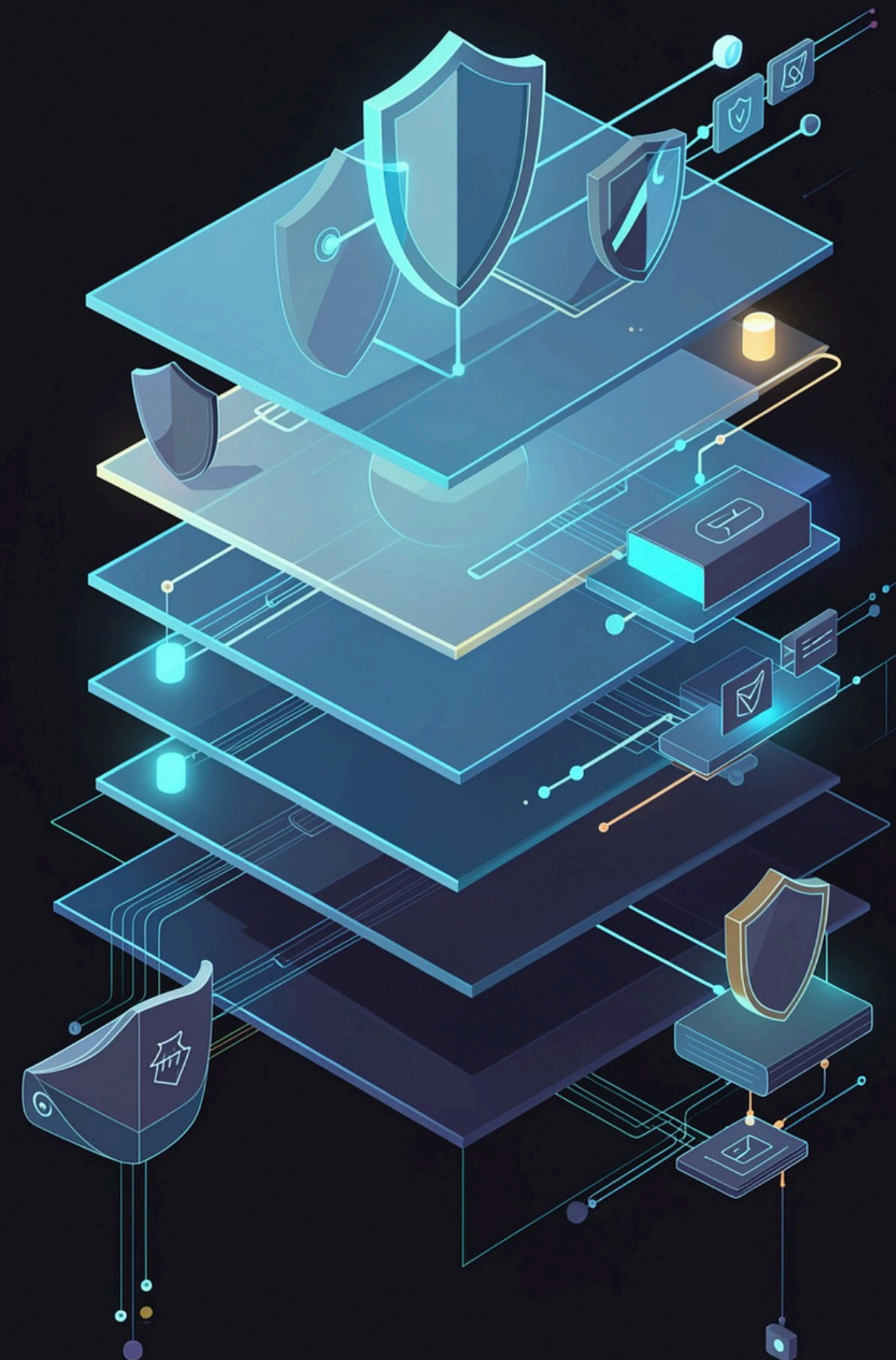
Protege la clave de 256 bits usando las credenciales del SO. Requiere dispositivo desbloqueado.

Capa 2 — SQLite3MultipleCiphers

Cifra y descifra automáticamente páginas de 4 KiB con ChaCha20-Poly1305. Drift trabaja con SQL normal.

Límites conocidos

Dato en RAM en claro mientras la app corre. Con dispositivo rooteado y desbloqueado, la clave es extraíble. No distingue usuarios del mismo dispositivo. TLS y autorización son capas aparte.



Conclusiones

Solución Integral Offline-First

Se blindó con éxito el working set clínico local de Tu Vacuna PAI frente a extracciones físicas y robos del dispositivo mediante el cifrado de archivos con SQLite3MultipleCiphers.

Transparencia Arquitectónica

El cifrado opera de forma nativa a nivel de almacenamiento sin alterar la lógica de negocio, las transacciones ni las consultas SQL ejecutadas por Drift.

Ingeniería Basada en Evidencia

La seguridad implementada no se basa en suposiciones teóricas; fue rigurosamente validada mediante guardas de regresión automatizadas y análisis empírico de bytes en disco (hexdump).

Madurez Técnica

Se estableció una distinción clara y honesta entre la protección en reposo lograda y las limitaciones operativas en memoria o dispositivos comprometidos, marcando la ruta para el hardening en producción.